

molchemist

Chemical structures and reactions in Typst

v0.1.6 2026-09-27

MIT AND BSD-3-Clause AND Apache-2.0 AND (Apache-2.0 WITH LLVM-exception)

Render chemical structures from Molfile, SDF, and SMILES data.

Eito Yoneyama

molchemist renders chemical structures from Molfile, SDF, SMILES, and MOL2 data. It also renders reactions from reaction SMILES or RXN files and R-group alternatives from RGfiles. The package preserves chemical metadata, generates 2D coordinates when needed, and draws figures through Alchemist and CeTZ.

The package is aimed at Typst documents that need compact molecule figures, publication-oriented skeletal formulae, and light annotation without leaving Typst.

Table of Contents

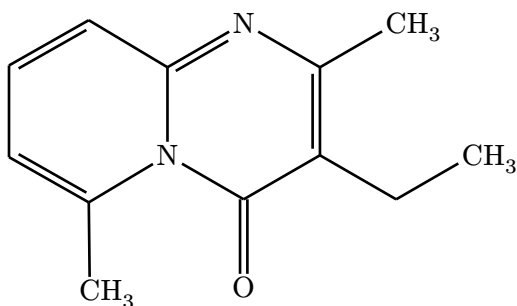
Getting Started	2	Index	48
Compatibility and Test Coverage	3		
Example Data	4		
Input and Semantic Fidelity	5		
Rendering Modes	22		
Coordinate Layout	23		
Styling	25		
Annotations	26		
Dump Mode	33		
Command-Line Export	34		
Publication Guidance	35		
SMILES Notes	36		
API Reference	38		
Limitations	46		
License and Dependencies	47		

Part I

Getting Started

Import `molchemist` and choose the renderer that matches your input: `#render-mol` for Molfile or SDF text, and `#render-smiles` for inline SMILES text.

```
#import "@preview/molchemist:0.1.6": *  
  
#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")  
#render-mol(mol-data, abbreviate: true)
```



The examples below assume this import. It also exports `cetz` for custom overlays.

I.1 Choosing an Input Format

- Molfile / SDF: coordinate-bearing structure text. Use it for database exports or drawing-tool output when preserving the supplied layout matters. Pass the one-based `{record}` option for multi-record SDF input. Typst 0.15.0 and later may pass `path("molecule.sdf")` directly; this manual uses `read(...)` for compatibility with earlier versions.
- SMILES: compact inline source text. Use it for examples, generated documents, and quick sketches. `molchemist` computes 2D coordinates before rendering.
- MOL2: Tripos molecule records with coordinates and atom types. Use `#render-mol2` or `#inspect-mol2`.
- Reaction SMILES / RXN: reactants, agents, and products. Use `#render-reaction` to compose the scheme and `#inspect-reaction` to inspect atom correspondence and bond changes.
- RGfile: a root structure and R-group alternatives. Use `#render-rgroup` for member panels and `#inspect-rgroup` for the source structure and logic.

Detailed guarantees and typeset examples for record selection, coordinate recovery, semantic inspection, CTfile queries and highlights, bond semantics, stereochemistry, and disconnected components are collected in **Input and Semantic Fidelity** rather than repeated in this introductory chapter.

Part II

Compatibility and Test Coverage

Surface	Minimum	Continuously tested
Typst package with <code>str</code> or <code>bytes</code> input	0.14.0	0.14.0, 0.14.1, 0.14.2, 0.15.0, and 0.15.1
Typst <code>path(...)</code> input	0.15.0	0.15.0 and 0.15.1
<code>molchemist-cli</code> build	Rust 1.86	Rust 1.86 on Ubuntu, macOS, and Windows

For each listed Typst version, CI compiles the rendering fixtures on Ubuntu and checks that package dump output is byte-for-byte identical to CLI output. Nine reviewed figures additionally have pixel comparisons on Typst 0.15.1 with bundled fonts. Rust parser, formatter, CLI, and WASM-host tests additionally run on macOS and Windows. The matrix covers the published package entrypoint and CLI-generated source, not the separate third-party toolchain used to typeset this manual.

Compatibility with Typst 0.14.0 and 0.14.1 is tested for users who cannot upgrade, but those releases contain an [upstream WebAssembly runtime security issue](#). Prefer Typst 0.14.2 or later for production documents.

Part III

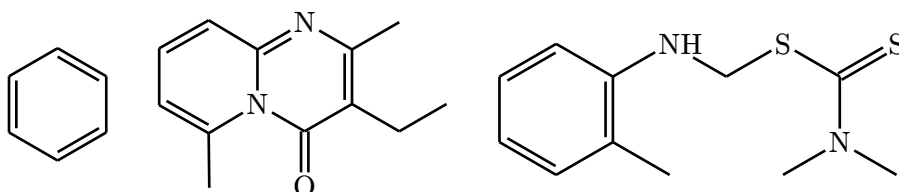
Example Data

The SDF examples in this manual use real PubChem records included in the repository test data.

- PubChem CID 241: benzene.
- PubChem CID 93406: 3-ethyl-2,6-dimethylpyrido[1,2-a]pyrimidin-4-one.
- PubChem SID 93298: a deposited DTP/NCI substance record associated with CID 235403.

```
#let benzene = read("Structure2D_COMPOUND_CID_241.sdf")
#let fused = read("Structure2D_COMPOUND_CID_93406.sdf")
#let deposited = read("DepositedStructure_SUBSTANCE_SID_93298_Version_3.sdf")

#grid(
  columns: 3,
  gutter: 7mm,
  align: center + horizon,
  render-mol(benzene, skeletal: true, config: (atom-sep: 1.55em)),
  render-mol(fused, skeletal: true, config: (atom-sep: 1.55em)),
  render-mol(deposited, skeletal: true, config: (atom-sep: 1.55em)),
)
```



Part IV

Input and Semantic Fidelity

The examples in this chapter focus on information that is easy to lose when a molecular file is reduced to a generic graph. Each example shows both the Typst source and the resulting drawing so that input selection, metadata, bond meaning, and stereochemistry can be checked independently.

IV.1 SDF Versions and Record Selection

`#render-mol` detects V2000 and V3000 independently for every selected SDF record. The `(record)` option is one-based and defaults to 1; it does not depend on the format of preceding records. This makes mixed-version SDF collections usable without preprocessing.

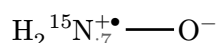
```
#let records = read("structures.sdf")

#grid(
  columns: 2,
  gutter: 10mm,
  align: center + top,
  [
    *V2000 · record 1*
    #v(2mm)
    #render-mol(records, record: 1)
  ],
  [
    *V3000 · record 2*
    #v(2mm)
    #render-mol(records, record: 2, abbreviate: true)
  ],
)
```

V2000 · record 1



V3000 · record 2



The second synthetic record also demonstrates V3000 charge, isotope, radical, and atom-map fields. Bond configuration is covered separately in the stereochemistry examples below. An out-of-range record number, an empty structure, malformed CTAB data, or non-finite coordinates raises an explicit error instead of silently drawing the wrong record.

IV.2 Semantic Inspection and Fidelity Policy

The `#inspect-mol` function exposes a versioned semantic dictionary before depiction. It contains the record header, stable source atom and bond IDs, enhanced stereo groups, SGroups and their attachment points, link nodes, collections, ordered SDF properties, the exact selected record, and diagnostics for parsed features the renderer cannot yet depict. V3000 source IDs retain their

original CTAB values; V2000 source IDs use one-based source positions. Unlike a dictionary, the property array preserves duplicate names and multiline values.

```
#let record = inspect-mol(read("structures.sdf"), record: 2)
#table(
  columns: 2,
  [Schema], [#record.schemaVersion],
  [Format], [#record.format],
  [First source ID], [#record.atoms.first().sourceId],
  [Properties], [#record.properties.len()],
  [Diagnostics], [#record.diagnostics.len()],
)
```

Schema	1
Format	molfile-v3000
First source ID	1
Properties	0
Diagnostics	0

Labelled, unexpanded multi-atom superatoms are contracted in every fidelity mode. SGroup SAP entries, variable-attachment bonds, link nodes, and atom/bond HILITE collections are preserved in the same semantic record; features without a standard or implemented depiction remain explicit diagnostics. Use `render-mol(data, fidelity: "strict")` to reject those gaps, including reaction-center flags and arbitrary user-defined collections. The default "ignore" renders the supported features. The command-line interface defaults to `--fidelity warn`; `molchemist inspect` writes the complete semantic record as JSON.

IV.3 Source Coordinates and Layout

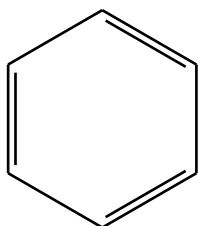
The default layout: "avoid" starts from usable source coordinates and increases spacing to clear measured atom labels while preserving angles and stereochemical orientation. Use layout: "coordinates" to retain supplied spacing, with components: "preserve" to retain relative component positions.

Coordgen generates new coordinates when the XY geometry is unusable—for example, when every bonded atom is collapsed onto one point—or when layout: "reflow" is selected. Automatic recovery retains source bond orders, wedge flags, atom metadata, and record ordering.

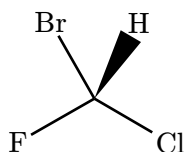
```
#let supplied = read("benzene-2d.sdf")
#let collapsed = read("collapsed-layout.sdf")

#grid(
  columns: 2,
  gutter: 12mm,
  align: center + top,
  [
    *Preserved 2D coordinates*
    #v(2mm)
    #render-mol(
      supplied,
      skeletal: true,
      config: (layout: "coordinates", components: "preserve"),
    )
  ],
  [
    *Recovered collapsed coordinates*
    #v(2mm)
    #render-mol(collapsed, skeletal: true)
  ],
)
```

Preserved 2D coordinates



Recovered collapsed coordinates



Spacing alone cannot separate coincident vertices or labels centered on unrelated bonds. Use `layout: "reflow"` or explicit atom positions for these cases. Set `collision-policy: "error"` to reject unresolved label collisions.

IV.4 Atom Metadata and Explicit Labels

Bracket-atom metadata remains structured through the SMILES parser, WASM boundary, formatter, and Typst renderer. Isotope numbers and formal charges use their conventional upper positions, hydrogen counts use a lower position, and atom classes remain visible as a small lower `:n` suffix. Invisible balancing attachments keep the primary atom symbols aligned when differently scripted labels appear in one disconnected structure.

```
#grid(
  columns: 3,
  gutter: 8mm,
  align: center + top,
  [
    *Atom class*
    #v(2mm)
    #render-smiles("[CH3:1]O", abbreviate: true)
  ],
  [
    *Isotope + class*
    #v(2mm)
    #render-smiles("[13CH3:7]C", abbreviate: true)
  ],
  [
    *Charge + H count*
    #v(2mm)
    #render-smiles("[NH4+].[Cl-]", skeletal: true)
  ],
)
```

Atom class	Isotope + class	Charge + H count	
H ₃ C _{:1} — OH	H ₃ ¹³ C _{:7} — CH ₃	NH ₄ ⁺	Cl ⁻

In full mode, explicitly represented hydrogen atoms remain separate graph nodes. In abbreviated and skeletal modes, foldable hydrogens join their parent labels, except when folding would erase a stereochemical wedge/dash or change the meaning of a bridging hydrogen.

IV.5 CTfile Queries, SGroups, Collections, and Attachments

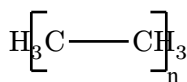
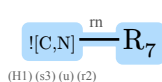
CTfile display information travels through the same semantic record and depiction AST. Atom lists use MDL-style glyphs such as ![C,N]. Hydrogen count, substitution count, unsaturation, ring-bond count, and valence constraints use compact parenthetical annotations such as (H1), (s3), (u), and (r2); set config: (ctfile: (query-details: true)) to show explicit labels such as implicit H ≥ 1 instead.

MDL query H0 means that no implicit hydrogen is allowed unless it is drawn explicitly; Hn means at least n implicit hydrogens in excess of hydrogens explicitly drawn. Consequently, V3000 HCOUNT=-1 normalizes to H0, while source values HCOUNT=1, 2, ..., 4 normalize directly to H1, H2, ..., H4; the source record remains available unchanged through [#inspect-mol](#). R-group labels remain visible, and ring/chain bond topology uses the plain rn / ch annotations. Labelled, unexpanded multi-atom SUP SGroups contract to graph nodes; expanded superatoms show their source atoms without duplicate group labels or brackets, and other known SGroup types use common brackets with adjacent labels. V3000 atom/bond MDLV30/HILITE collections render on

a background layer that follows glyph bounds. Ordered SDF properties remain available through inspection.

For substitution and ring-bond counts, the CTfile sentinel -2 is depicted as (s*) / (r*) (“as drawn”), and -1 as (s0) / (r0). Substitution values of six or more share the (s6) bucket; ring-bond values of four or more share (r4).

```
#let data = read("ctfile-fidelity.sdf")
#render-mol(
  data,
  fidelity: "strict",
  config: (
    ctfile: (
      highlight-paint: rgb("#74c0fc").transparentize(45%),
    ),
  ),
)
```

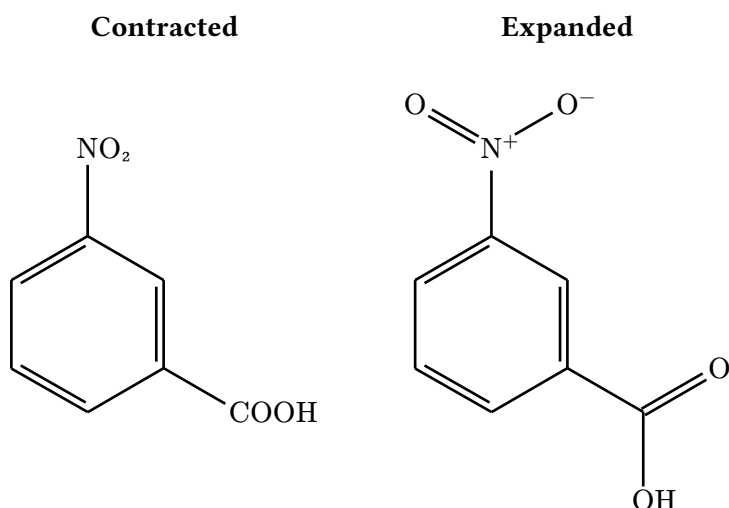


IV.5.1 Multi-atom Superautom Contraction

An unexpanded SUP SGroup with more than one source atom contracts to one labelled graph node. Internal atoms and bonds remain available through `inspect-mol`, while the depiction removes internal bonds and reconnects every crossing bond to the contracted glyph. With one attachment atom, its source coordinate is retained; with several attachment atoms, their centroid defines the glyph position. Atom highlights on hidden members project to the glyph, crossing-bond highlights follow the reconnected bond, and an internal-bond-only highlight also resolves to the glyph.

The two panels use the same ACD/Labs V2000 N02 / C00H fixture distributed by RDKit. The default contracts labelled SGroups, while `sgroups: "expanded"` draws their source atoms and bonds. Both views pass strict fidelity and align the same scaffold atom with `baseline-atom: "a0"`; inspection retains the original CTfile labels and membership.

```
#let data = read("assets/Sgroups_Abbreviations.mol")
#context {
  let first = render-mol(data, skeletal: true, fidelity: "strict",
    config: (baseline-atom: "a0"))
  let second = render-mol(data, skeletal: true, fidelity: "strict",
    config: (sgroups: "expanded", baseline-atom: "a0"))
  let first-width = measure(first).width
  let second-width = measure(second).width
  let width = calc.max(first-width, second-width)
  box(width: 2 * width + 10mm, {
    grid(columns: (width, width), column-gutter: 10mm, align: center,
      [*Contracted*], [*Expanded*])
    v(3mm)
    h((width - first-width) / 2)
    first
    h(width - (first-width + second-width) / 2 + 10mm)
    second
    h((width - second-width) / 2)
  })
}
```



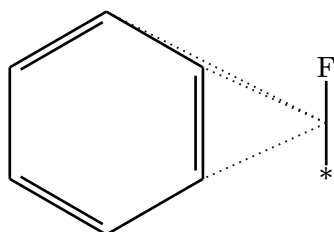
Set config: (sgroups: "expanded") to display the full source graph of SUP groups, including groups whose labels are missing or whose membership overlaps. Inspection retains the original input. HILITE selections of known SGroups highlight their member atoms and bonds in the expanded view. The CLI equivalent is `--expand-superatoms`.

IV.5.2 Attachment Points, Link Nodes, and Collections

SGroup M SAP and SAP=(...) entries remain structured as attachment atom, optional leaving atom, and connection ID; contracted multi-atom superatoms use those explicit attachment atoms to choose the glyph position. V2000 M AP0 and V3000 ATTCHPT are R-group-member attributes, not main-CTAB atom decorations. A value found on the main CTAB is therefore preserved but rejected by strict fidelity instead of being presented as a standard glyph.

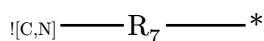
V2000 M LIN and V3000 LINKNODE entries render their minimum–maximum repeat range next to the link atom. A V3000 variable-attachment bond preserves ENDPTS and ATTACH: ANY alternatives use dotted branches, while ALL uses solid branches. Atom lists belong to atom types, and R-group numbers belong to RGROUPS; they are not COLLECTION kinds. MDLV30/HILITE has a defined renderer convention. Other internal or user-defined collections retain their full name, atom/bond/SGroup/3D/R-group member lists, generic members, and original logical entry through `#inspect-mol`, but no visual convention is invented for them.

```
#let data = read("ctfile-attachments-collections.sdf", encoding: none)
#align(center)[
  #render-mol(
    data, record: 1, skeletal: true, fidelity: "strict",
    config: (atom-sep: 3.8em),
  )
]
```



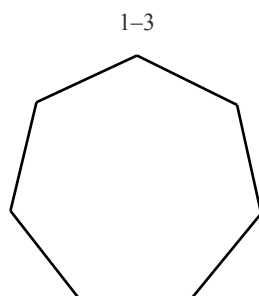
V3000 position variation: dotted branches are the ATTACH=ANY endpoint alternatives.

```
#let data = read("ctfile-attachments-collections.sdf", encoding: none)
#align(center)[
  #render-mol(
    data, record: 2, skeletal: true, fidelity: "strict",
    config: (atom-sep: 3.8em),
  )
]
```



Standard atom-list query and R-group 7 atom attributes; the terminal * is an explicit wildcard atom.

```
#let data = read("Sgroups_Link_01.mol", encoding: none)
#align(center)[
  #render-mol(
    data, skeletal: true, fidelity: "strict",
    config: (atom-sep: 3.8em),
  )
]
```



Real V2000 M LIN fixture from RDKit; the link repeat range is 1-3.

```
#let data = read("ctfile-attachments-collections.sdf", encoding: none)
#let record = inspect-mol(data, record: 3)
#let collection = record.collections.first()
#table(
  columns: 2,
  [Name], [#raw(collection.name)],
  [Kind], [#collection.kind],
  [Atom IDs], [#collection.atomSourceIds.map(str).join(", ")],
  [Bond IDs], [#collection.bondSourceIds.map(str).join(", ")],
  [Diagnostic], [#record.diagnostics.first().code],
)
```

Name	molchemist/SELECTION
Kind	user-defined
Atom IDs	10
Bond IDs	7
Diagnostic	user-defined-collection-not-depicted

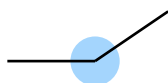
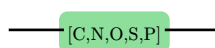
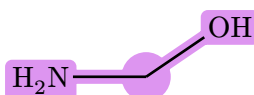
IV.5.3 Highlight Depiction

V3000 MDLV30/HILITE collections keep atom and bond membership separate. A highlighted skeletal carbon uses a circle centered on the hidden vertex. A visible atom uses the same visual treatment, expanding to a rounded rectangle only when its rendered glyph bounds are wider than the configured radius. Bond-only highlights use a constant-width capsule with round ends.

Atom and bond subpaths share one non-zero compound fill, so connected selections read as one continuous region while disconnected selections remain visually separate.

```
#let data = read("highlight-shapes.sdf", encoding: none)
#let highlight-case(title, record, paint) = align(center)[
  *#title*
  #v(2mm)
  #render-mol(
    data, record: record, skeletal: true, fidelity: "strict",
    config: (ctfile: (
      highlight-paint: paint.transparentize(35%),
    )),
  )
]

#grid(
  columns: 2,
  gutter: 10mm,
  row-gutter: 8mm,
  align: top,
  highlight-case([Atom only · skeletal vertex], 1, rgb("#74c0fc")),
  highlight-case([Atom only · query glyph], 2, rgb("#51cf66")),
  highlight-case([Bond only · capsule], 3, rgb("#ffd43b")),
  highlight-case([Connected atom + bond], 4, rgb("#cc5de8")),
  highlight-case([Disconnected regions], 5, rgb("#ff922b")),
)
```

Atom only · skeletal vertex**Atom only · query glyph****Bond only · capsule****Connected atom + bond****Disconnected regions**

The regression corpus also includes [RDKit/Bingo's real V3000 v3k.crash1.mol](#), which declares the same source ID in separate atom and bond HILITE collections. Semantic tests preserve those source memberships independently, and the SVG regression checks the compound-path topology for every case above.

IV.6 Bond Semantics

V3000 bond orders 4 through 10 remain distinct: aromatic, single-or-double, single-or-aromatic, double-or-aromatic, any, coordination, and hydrogen bonds are not collapsed to ordinary single bonds. Query and aromatic bonds use dashed/dotted partial lines, any/either bonds use a wavy line, coordination bonds preserve their donor-to-acceptor direction with a filled arrowhead, and hydrogen bonds use a dotted line. A SMILES \$ bond is rendered separately as four parallel lines.

V3000 order	Meaning	Visual cue
4	Aromatic	Double line with a dashed partial line
5	Single or double	Double line with a dotted partial line
6	Single or aromatic	Dashed single line
7	Double or aromatic	Dashed double line
8	Any	Wavy line
9	Coordination	Directed line with a filled arrowhead
10	Hydrogen	Dotted line

```
#let extended = read("bond-semantics.sdf")

#grid(
  columns: 1,
  row-gutter: 5mm,
  align: center,
  [
    *V3000 extended bond orders*
    #v(2mm)
    #render-mol(
      extended,
      abbreviate: true,
      config: (atom-sep: 3.0em),
    )
  ],
  [
    *OpenSMILES quadruple bond*
    #v(2mm)
    #render-smiles("[Cr]${[Cr]}")
  ],
)
```

V3000 extended bond orders

=====N.....O----S====P~~~~~F→Cl.....Br~~~~~I

OpenSMILES quadruple bond

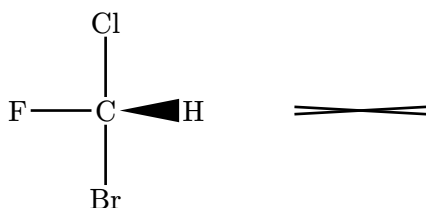
Cr≡≡≡Cr

Long hydrogen bonds are excluded when the renderer computes a representative covalent bond length. This prevents a distant noncovalent contact from shrinking the covalent part of the structure.

IV.7 SDF Stereochemical Metadata

For Molfile/SDF input, up/down single bonds remain wedge/dash bonds. Undefined double-bond geometry—V2000 stereo code 3 or V3000 double-bond CFG=2—uses a crossed double bond. Atom CFG parity and V3000 STEABS, STEREL, and STERAC collections are retained as annotations rather than being discarded.

```
#let stereo-data = read("stereochemistry.sdf")  
#render-mol(stereo-data, skeletal: true)
```



Stereo annotations: C CFG=1; OR7 (a0)

The explicit hydrogen above is intentionally kept visible because its wedge carries stereochemical information. The disconnected crossed double bond in the same record also demonstrates that component separation does not drop bond configuration.

IV.7.1 Projection from 3D Coordinates

Set `config: (infer-stereo: true)` to project nondegenerate 3D coordinates into a generated 2D layout. Near-planar double-bond substituent geometry is transferred to Coordgen; tetrahedral wedges preserve signed volume. Three-ligand neutral carbon or silicon centers can use an implicit fourth hydrogen. Other centers require four explicit ligands. Explicit stereochemical input takes precedence.

```
#render-mol(  
  read("structure-3d.mol"),  
  skeletal: true,  
  config: (infer-stereo: true),  
)
```

The projection determines local wedge orientation. It does not assign CIP R/S descriptors or assess conformational stability. Planar or degenerate coordinates and indistinguishable ligands do not produce inferred tetrahedral stereochemistry. Invalid projections are rejected; non-tetrahedral stereo requires an explicit SMILES or CTfile description. The CLI equivalent is `--infer-stereo`.

IV.8 SMILES Tetrahedral and Double-Bond Stereo

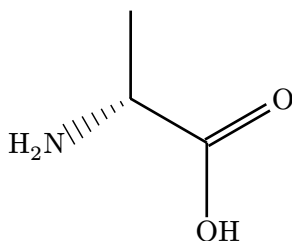
Stereo

OpenSMILES @ and @@ are interpreted using local SMILES neighbor order, not by assigning a fixed wedge to one token. Branch order, bracket hydrogens, incoming atoms, and ring-closure token positions therefore contribute to the depicted configuration. Directional / and \ bonds are likewise resolved as a pair around the double bond.

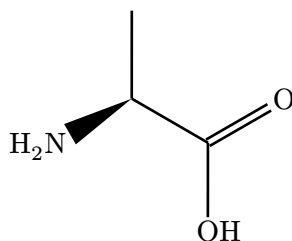
```
#let stereo-example(title, source) = align(center)[
  *#title*
  #v(2mm)
  #render-smiles(source, skeletal: true)
]

#grid(
  columns: 2,
  gutter: 10mm,
  row-gutter: 6mm,
  align: top,
  stereo-example([D-alanine · (R)], "N[C@H](C)C(=O)O"),
  stereo-example([L-alanine · (S)], "N[C@@H](C)C(=O)O"),
  stereo-example([E-difluoroethene], "F/C=C/F"),
  stereo-example([Z-difluoroethene], "F/C=C\F"),
)
```

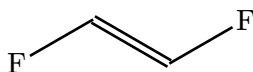
D-alanine · (R)



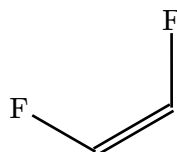
L-alanine · (S)



E-difluoroethene



Z-difluoroethene



The implicit-hydrogen form N[C@@H](C)C(=O)O and the explicit-hydrogen form N[C@@]([H])(C)C(=O)O describe the same local configuration and are tested to produce the same absolute orientation.

IV.9 Multi-component Structures

Disconnected Molfile/SDF graphs and dot-separated SMILES retain every component in source order. Components are separated by whitespace with no visible synthetic operator. Atom and bond indices remain global, so annotation anchors can target later components without renumbering them.

```
#grid(
  columns: 1,
  row-gutter: 5mm,
  align: center,
  [
    *Dot-separated salt*
    #v(2mm)
    #render-smiles("[Na+].[Cl-]", abbreviate: true)
  ],
  [
    *Visible isolated components*
    #v(2mm)
    #render-smiles("[H+].C.[Cl-]", skeletal: true)
  ],
)
```

Dot-separated salt



Visible isolated components



Isolated hydrogen and zero-heavy-neighbor carbon components remain visible in abbreviated and skeletal modes. Scripted labels are balanced around their primary symbols, so H, CH, and Cl align while the 4 in CH₄ remains correctly lowered.

IV.10 Tripos MOL2

`#render-mol2` and `#inspect-mol2` accept one-based record selection for Tripos MOL2 input. Atom IDs, coordinates, and supported bond types are converted to V3000 for rendering and inspection.

```
#render-mol2(read("molecule.mol2"), record: 1, skeletal: true)
#let record = inspect-mol2(read("molecule.mol2"), record: 1)
```

The exact selected MOL2 body remains as a JSON string in the MOL2_SOURCE property, including substructures, source atom types, partial charges, blank lines, and original line endings. Partial

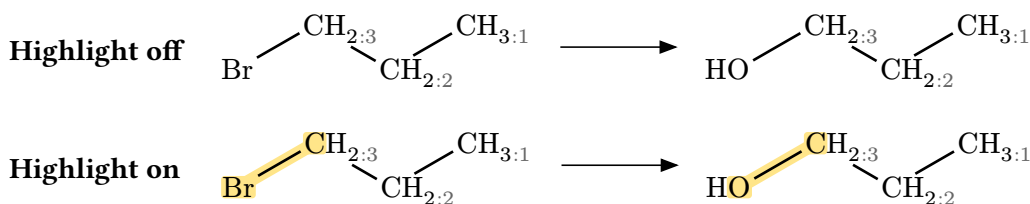
charges are retained without rounding them into formal charges. The explicit N.4 atom type supplies its formal positive charge. Unknown atom and bond types are rejected. An nc bond creates no graph edge; du and un retain query-bond behavior.

IV.11 Reactions and Atom Correspondence

`#render-reaction` accepts reaction SMILES and RXN V2000/V3000 input. It places reactants and products around an arrow, with agents and optional conditions. Changed bonds and their atoms, including entering and leaving groups, are highlighted by default.

Both rows show the same mapped conversion of 1-bromopropane to 1-propanol. Only highlight-center changes. The broken C-Br bond and the formed C-O bond are highlighted along with their endpoint element symbols. Atom and bond backgrounds join across the small gap reserved between the bond line and the element symbol. The C-C backbone, hydrogen labels, isotopes, and atom-map numbers remain unhighlighted.

```
#let reaction = "[CH3:1][CH2:2][CH2:3]Br>>[CH3:1][CH2:2][CH2:3]O"
#grid(
  columns: 2, column-gutter: 5mm, row-gutter: 6mm,
  align: (left + horizon, left + horizon),
  [*Highlight off*], render-reaction(reaction, highlight-center: false),
  [*Highlight on*], render-reaction(reaction, highlight-center: true),
)
```



Use format: "rxn" for an RXN file and highlight-center: false to disable reaction-center highlighting.

```
#render-reaction(read("reaction.rxn"), format: "rxn")
#let analysis = inspect-reaction(
  "CCO>>CC=O",
  infer-mapping: true,
  search-limit: 200000,
)
```

Inspection returns reactants, agents, products, atom correspondences, broken/formed/order-changed bonds, unmatched atoms, ambiguity, and search completion. Rendering retains the same analysis in Typst metadata with kind: "molchemist-reaction".

Supplied atom-map numbers constrain correspondence. Optional inference searches element/isotope-compatible assignments, maximizing the number of matched atoms first and conserved bond orders second. Equivalent best assignments set `ambiguous`; exhausting the search budget sets `searchComplete: false`. Use `mapping-policy: "strict"` to reject either case. `fidelity: "strict"` additionally checks CTfile depiction diagnostics in every component.

Correspondence describes the supplied graphs, rather than predicting a reaction mechanism. Bond changes use the supplied bond-order representation, so aromatic and Kekule forms should be normalized consistently by the source application.

IV.12 R-group Alternatives

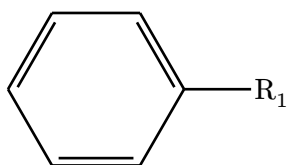
`#inspect-rgroup` and `#render-rgroup` accept V2000 RGfiles and V3000 root/member CTAB blocks. The figure shows the root and every member alternative. Numbered asterisks identify external attachment sites; occurrence, remaining-site, and dependency conditions are presented as text.

The synthetic RGfile below supplies the same aromatic root to both panels. The right panel also displays its methoxy and cyano alternatives, their attachment sites, and the interpreted occurrence condition.

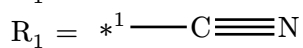
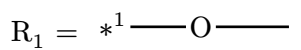
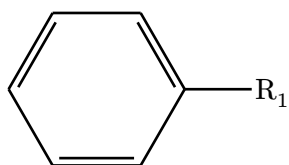
Inspection preserves `rawSource`, raw logic lines, normalized conditions, and all member alternatives. Rendering displays the alternatives without selecting a member or enumerating all molecules satisfying the Markush query. R-group references within members remain explicit. ATTCHPT and M AP0 are interpreted in member context; unsupported member features are rejected by the default `fidelity: "strict"`.

```
#let data = read("assets/rgroup-alternatives.mol")
#let groups = inspect-rgroup(data)
#let root = render-mol(groups.root, skeletal: true)
#let alternatives = render-rgroup(data)
#grid(
  columns: 2, gutter: 10mm, align: center + top,
  [*Root only* #v(3mm) #root],
  [*Root and alternatives* #v(3mm) #alternatives],
)
```

Root only



Root and alternatives



R₁: at least one site; remaining sites: H.

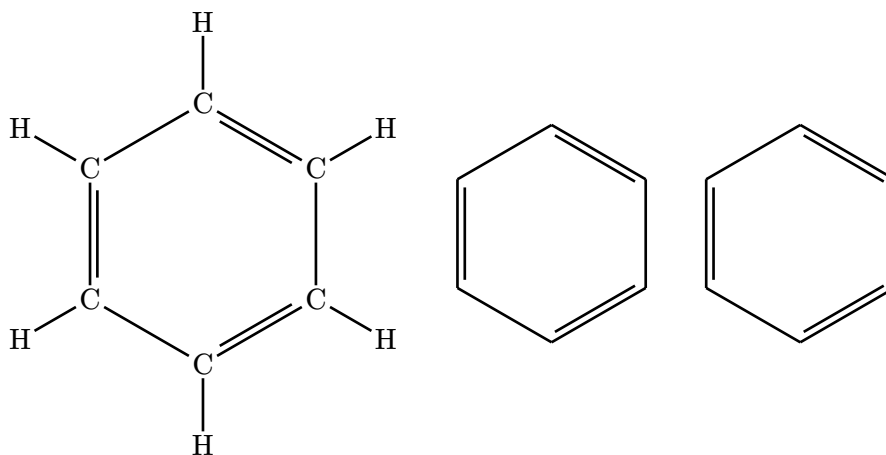
Part V

Rendering Modes

Both renderers support the same three modes. Full mode is useful for small molecules and debugging. Abbreviated mode folds common hydrogens and terminal carbons into labels. Skeletal mode is usually the best default for publication figures.

```
#let mol-data = read("Structure2D_COMPOUND_CID_241.sdf")

#grid(
  columns: 3,
  gutter: 8mm,
  align: center + horizon,
  render-mol(mol-data),
  render-mol(mol-data, abbreviate: true),
  render-mol(mol-data, skeletal: true),
)
```



Part VI

Coordinate Layout

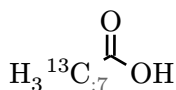
The renderer places vertices explicitly and clips bonds to label bounds measured in the document font. It uses Alchemist 0.2.0 and its exported CeTZ module.

config.layout	Behavior
"coordinates"	Retain supplied spacing and clip bonds at labels.
"avoid" (default)	Increase spacing to clear labels from other labels and unrelated bonds, up to max-layout-scale.
"reflow"	Generate fresh SDF coordinates with Coordgen, then apply the spacing pass. SMILES already generates coordinates.

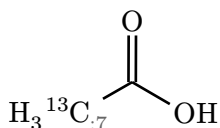
Both panels use the same isotope-labelled molecule, font size, and atom-sep: 1.2em. The left intentionally shows crowded source spacing; "avoid" clears the label collisions by increasing coordinate spacing.

```
#let molecule = "[13CH3:7]C(=O)O"
#context {
  let first = render-smiles(molecule, abbreviate: true,
    config: (layout: "coordinates", atom-sep: 1.2em))
  let second = render-smiles(molecule, abbreviate: true,
    config: (layout: "avoid", atom-sep: 1.2em))
  let first-width = measure(first).width
  let second-width = measure(second).width
  let width = calc.max(first-width, second-width)
  box(width: 2 * width + 10mm, {
    grid(columns: (width, width), column-gutter: 10mm, align: center,
      [*Coordinates*], [*Avoid*])
    v(3mm)
    h((width - first-width) / 2)
    first
    h(width - (first-width + second-width) / 2 + 10mm)
    second
    h((width - second-width) / 2)
  })
}
```

Coordinates



Avoid



Spacing changes preserve angles and stereochemical orientation. They do not move an individual stereocenter independently. `reflow` preserves defined double-bond geometry and recomputes explicit tetrahedral wedge orientation after changing the XY geometry.

Molfile atoms receive implicit hydrogen labels when the supplied valence and bond orders determine the count for supported main-group elements. Explicit H atoms, charges, radicals, and zero-valence declarations are respected. Query hydrogen counts, aromatic or unknown bond orders, and external R-group attachment sites do not supply an inferred H count.

VI.1 Collision Handling

`collision-gap` sets clearance in units of `atom-sep` (default 0.12). `max-layout-scale` limits coordinate enlargement (default 16). `min-bond-length` sets the minimum visible bond segment in `atom-sep` units; its default is the larger of 0.45 units and 1em. Clipping at both endpoint labels is included in this check. `collision-policy`: "error" rejects unresolved atom-label collisions, short or hidden bonds, and unplaced automatic callouts. The default "report" records them in Typst metadata with kind: "molchemist-layout", including scale, collisions, and unplaced-callouts.

Atom labels are anchored on the bonded element symbol. Hydrogen labels face away from neighboring bonds when possible. Collision checks cover atom labels against labels and unrelated bonds, as well as visible bond lengths. Arbitrary CeTZ overlays, SGroup brackets, and query-detail labels require final figure review. Coincident vertices and labels centered on unrelated bond axes may require `reflow` or explicit coordinates because scaling cannot separate them.

VI.2 Components and Manual Positions

`components`: "pack" places disconnected components side by side by default. Use `components`: "preserve" to retain source component positions. `baseline-atom` selects a depiction atom such as "a0" as the inline baseline; reaction panels otherwise align the vertical center of their graph coordinates. Highlight bounds do not change this baseline. `atom-positions` maps depiction names such as "a2" to (x, y) in `atom-sep` units; packing translates components after applying these overrides.

Use `layout`: "coordinates" with `components`: "preserve" to keep manual positions fixed:

```
#render-mol(read("molecule.sdf"), config: (  
  layout: "coordinates",  
  components: "preserve",  
  atom-positions: (a2: (1.5, 0.8)),  
))
```

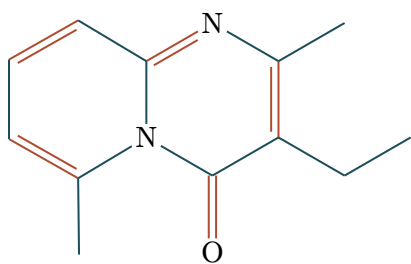
Part VII

Styling

Pass `<config>` for visual adjustments that should reach `alchemist`, such as atom spacing, fragment color, and bond styles.

```
#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")

#render-mol(
  mol-data,
  skeletal: true,
  config: (
    atom-sep: 2.6em,
    fragment-color: rgb("#1b4d5a"),
    single: (stroke: 0.75pt + rgb("#1b4d5a")),
    double: (stroke: 0.75pt + rgb("#b14b2f")),
  ),
)
```



Use `atom-sep`, `font`, and `stroke` settings for visual tuning. The default layout: "avoid" adjusts spacing to measured labels; layout: "reflow" generates new coordinates. Alchemist's routing settings do not control this coordinate renderer.

Part VIII

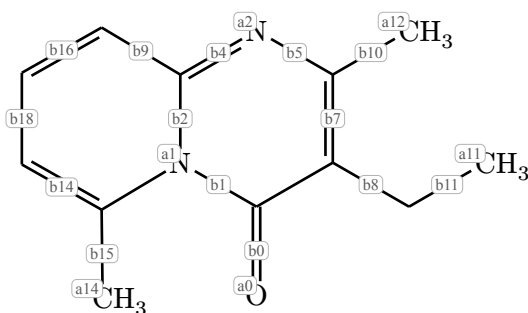
Annotations

Annotations are drawn after the molecule. They are meant for sparse publication callouts, not for replacing a full figure editor.

VIII.1 Finding Atom and Bond Indices

Enable `(show-indices)` while authoring. The visible labels are the indices used by `#atom-anchor` and `#bond-anchor`.

```
#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")  
#render-mol(mol-data, abbreviate: true, show-indices: true)
```



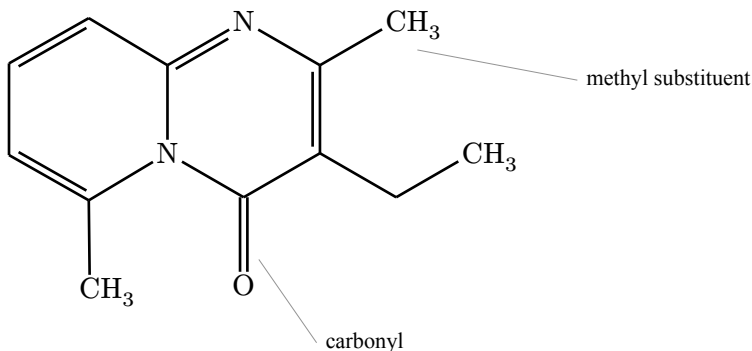
VIII.2 Callouts

Use `#callout-annotation` for external labels. Defaults are intentionally quiet: no arrowhead, a thin leader line, and an unboxed label.

When `label-at` is `auto`, the renderer searches for free positions for atom/bond callouts and routes straight or orthogonal leaders around obstacles. Explicit positions are respected. Set `config: (auto-annotations: false)` to use placement based on the requested side. Unplaced automatic callouts are included in layout metadata; `collision-policy: "error"` rejects them.

```
#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")

#render-mol(
  mol-data,
  abbreviate: true,
  annotations: (
    callout-annotation(
      bond-anchor(0),
      [carbonyl],
      label-at: (to: molecule-anchor(anchor: "south"), rel: (0.82, -0.48)),
      leader-start: (to: molecule-anchor(anchor: "south"), rel: (0.7, -0.48)),
      leader-end: (to: bond-anchor(0), rel: (0.2, -0.24)),
      leader: "curve",
      stroke: luma(58%) + 0.28pt,
      label-size: 0.76em,
      label-anchor: "west",
    ),
    callout-annotation(
      atom-anchor(12),
      [methyl substituent],
      label-at: (to: molecule-anchor(anchor: "east"), rel: (0.9, 1.04)),
      leader-start: (to: molecule-anchor(anchor: "east"), rel: (0.78, 1.04)),
      leader-end: (to: atom-anchor(12), rel: (0.28, -0.38)),
      leader: "curve",
      stroke: luma(58%) + 0.28pt,
      label-size: 0.76em,
      label-anchor: "west",
    ),
  ),
),
)
```

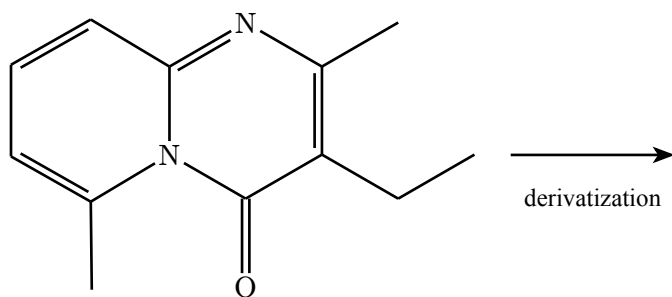


VIII.3 Arrows

Use `#arrow-annotation` for molecule-level process arrows or simple directional marks.

```
#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")

#render-mol(
  mol-data,
  skeletal: true,
  annotations: arrow-annotation(
    (to: molecule-anchor(anchor: "east", rel: (0.48, 0)),
    (to: molecule-anchor(anchor: "east", rel: (2.65, 0)),
    label: [derivatization],
    label-offset: (0, -0.46),
    label-anchor: "north",
  ),
)
```

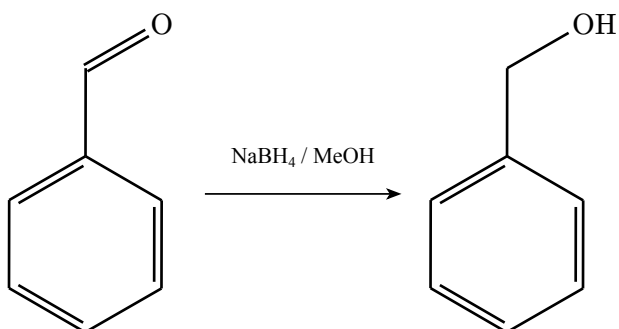


VIII.4 Custom Reaction Schemes

Use `#render-reaction` for reaction SMILES or RXN input. To customize the arrangement or draw a mechanism, compose separate molecule renderings with Typst and CeTZ. This gives direct control over reaction arrows and their labels.

```
#let reaction-arrow = cetz.canvas({
  import cetz.draw: *
  line(
    (0.1, 0), (2.7, 0),
    stroke: 0.65pt + black,
    mark: (end: ">>", scale: 0.72, fill: black),
  )
  content((1.4, 0.4), text(size: 0.76em)[NaBH4 / MeOH], anchor: "south")
})

#grid(
  columns: (auto, 26mm, auto),
  column-gutter: 4mm,
  align: horizon + center,
  render-smiles("O=CC1=CC=CC=C1", abbreviate: true),
  reaction-arrow,
  render-smiles("OCC1=CC=CC=C1", abbreviate: true),
)
```

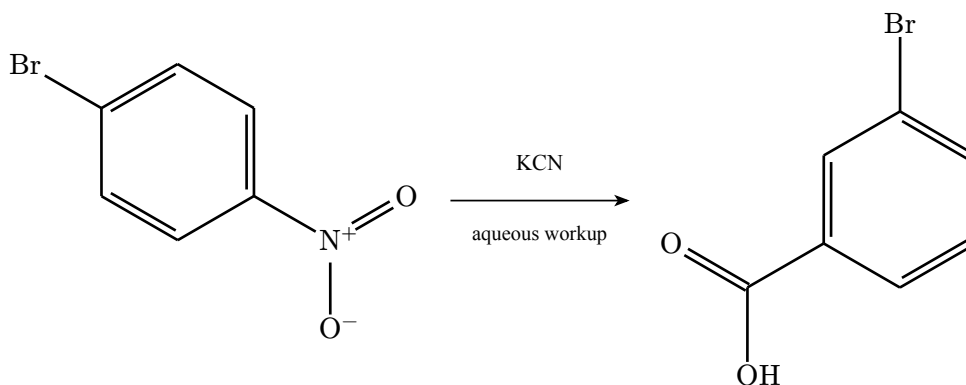


VIII.5 Schematic von Richter Transformation

Use `render-reaction` for RXN and Reaction SMILES input with graph-edit highlights. Separate molecule renderings remain useful when manually composing a mechanism or a custom reaction scheme. The net conversion below is a schematic overview, not an exhaustive electron-pushing mechanism: cyanide addition and rearrangement are represented by the labelled arrow, and the product is shown after aqueous workup.

```
#let scheme-arrow(above, below) = cetz.canvas({
  import cetz.draw: *
  line(
    (0.08, 0), (2.45, 0),
    stroke: 0.65pt + black,
    mark: (end: ">", scale: 0.72, fill: black),
  )
  content((1.26, 0.4), text(size: 0.76em)[#above], anchor: "south")
  content((1.26, -0.34), text(size: 0.7em)[#below], anchor: "north")
})

#grid(
  columns: (auto, 24mm, auto),
  column-gutter: 4mm,
  align: horizon + center,
  render-smiles("O=[N+](O-)c1ccc(Br)cc1", skeletal: true),
  scheme-arrow([KCN], [aqueous workup]),
  render-smiles("O=C(O)c1ccc(Br)cc1", skeletal: true),
)
```



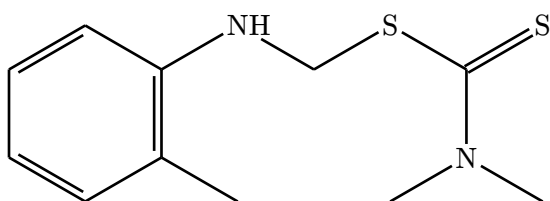
This transformation is discussed by M. Rosenblum, *The Mechanism of the von Richter Reaction*, J. Am. Chem. Soc. 82 (1960), 3796–3798 (DOI 10.1021/ja01499a090). A detailed mechanistic figure should cite a chosen mechanistic model and draw its individual intermediates explicitly.

VIII.6 Low-Level Labels

Use `#label-annotation` when no leader line is needed.

```
#let mol-data = read("DepositedStructure_SUBSTANCE_SID_93298_Version_3.sdf")

#render-mol(
  mol-data,
  skeletal: true,
  annotations: label-annotation(
    molecule-anchor(anchor: "south"),
    [PubChem SID 93298],
    offset: (0, -0.65),
    label-anchor: "north",
  ),
)
```



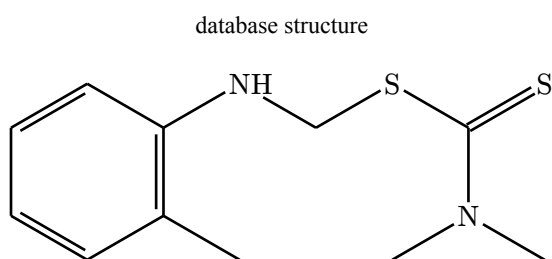
PubChem SID 93298

VIII.7 Custom CeTZ Overlays

For final figure polishing, `#cetz-annotation` exposes the generated molecule name, so you can draw directly against CeTZ anchors.

```
#let mol-data = read("DepositedStructure_SUBSTANCE_SID_93298_Version_3.sdf")

#render-mol(
  mol-data,
  skeletal: true,
  annotations: cetz-annotation(mol => {
    import cetz.draw: *
    content(
      (to: (name: mol, anchor: "north"), rel: (0, 0.54)),
      text(size: 0.82em)[database structure],
      anchor: "south",
    )
  }),
)
```



Part IX

Dump Mode

With `(dump)`, `molchemist` returns generated `alchemist` source instead of a drawing. Use this when a figure needs manual surgery beyond the annotation API.

```
#let mol-data = read("Structure2D_COMPOUND_CID_241.sdf")
#render-mol(mol-data, skeletal: true, dump: true)
```

The returned source embeds the drawing modules followed by `_scene` and `_scene-config`. Export it with the CLI to inspect or edit the complete program.

Exported configuration values may contain Typst lengths and colors. Configuration callbacks cannot be serialized. Font measurement runs when the generated source is compiled.

Part X

Command-Line Export

For scripts and editor workflows, install the `molchemist-cli` crate with `cargo install --locked molchemist-cli`. Its `molchemist dump` command accepts Molfile, SDF, SMILES, MOL2, reaction SMILES, RXN, and RGfile input and writes the same formatted source as `(dump)` to standard output. Add `--standalone` to set an auto-sized page, or `--output figure.typ` to write directly to a file.

Generated source contains an Alchemist import, the drawing helpers, a serialized `_scene`, and its `_scene-config`. Font measurement runs when Typst compiles this source. The scene and configuration can be edited without depending on the `molchemist` package.

```
$ molchemist dump molecule.sdf > molecule.typ
$ molchemist dump structures.sdf --record 2 --mode skeletal > record-2.typ
$ molchemist dump --smiles 'CC(=O)O' --mode skeletal --standalone --output
acetic-acid.typ
$ typst compile acetic-acid.typ
```

Input may also be piped from another program—for example, `printf '%s\n' 'c1ccccc1' | molchemist dump --format smiles > benzene.typ`. Diagnostics are written to standard error and generated source exclusively to standard output, so redirection does not mix warnings or errors into a `.typ` file.

The CLI detects `.mol2` and `.rxn` files and recognizes R-group records. Use `--format mol2`, `--format rxn`, or `--format rgroup` to select the format explicitly.

```
molchemist dump molecule.sdf --layout avoid --strict-collisions --
standalone
molchemist dump molecule.sdf --layout reflow --components preserve --
standalone
molchemist dump structure-3d.mol --infer-stereo --standalone
molchemist dump groups.mol --expand-superatoms --standalone
molchemist dump molecule.mol2 --mode skeletal --standalone
molchemist dump alternatives.mol --format rgroup --fidelity strict
molchemist dump reaction.rxn --conditions oxidation --standalone
molchemist inspect --text 'CCO>>CC=O' --infer-mapping
```

`--strict-collisions` rejects unresolved collisions when the exported Typst source is compiled. For reactions, `--fidelity strict` rejects ambiguous or incomplete atom correspondence as well as unsupported CTfile features. Use `--mapping-search-limit` to change the inference budget.

Part XI

Publication Guidance

For paper figures, start with `<skeletal>` for hydrocarbon-heavy structures and `<abbreviate>` when heteroatom hydrogens or terminal groups should remain explicit. Keep full mode for small structures and debugging.

Keep annotations sparse. In most cases, a thin unboxed `#callout-annotation` is better than a boxed label or an arrowhead. If a leader line looks like a chemical bond, move the label with `<label-at>` or stop the leader outside the structure with `<leader-end>`.

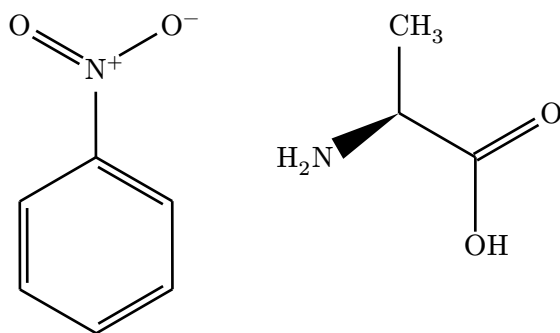
Dense structures can still overlap in full mode, especially after SMILES implicit hydrogens are expanded. Prefer abbreviated or skeletal mode when a molecule is meant for a publication figure.

Part XII

SMILES Notes

SMILES support is a parse-and-layout pipeline. The parser accepts common SMILES notation, aromatic rings, charges, tetrahedral @ / @@ centers, and / / \ double-bond geometry. It rejects malformed branch, dot, bond, bracket-property, charge, isotope, atom-class, directional-bond, and aromatic notation instead of normalizing it silently. Atom classes from 0 through 9999 are accepted, and aromatic systems must satisfy Hückel's rule and admit a valence-compatible Kekulé assignment. Tetrahedral centers retain OpenSMILES local neighbor order, including bracketed hydrogens and ring-closure token positions.

```
#grid(  
  columns: 2,  
  gutter: 10mm,  
  align: top + center,  
  render-smiles("O=[N+](O-)c1ccccc1", abbreviate: true),  
  render-smiles("N[C@@H](C)C(=O)O", abbreviate: true),  
)
```



Extended OpenSMILES chirality classes are rendered geometrically when their topology permits an unambiguous projection. @AL uses terminal wedge/dash bonds, @SP uses its U/4/Z ligand path, and @TB / @OH combine the specified ligand winding with solid and hashed viewing-axis bonds. Invalid or cyclic topologies that cannot be rearranged safely retain their original chirality tag as a fallback annotation.

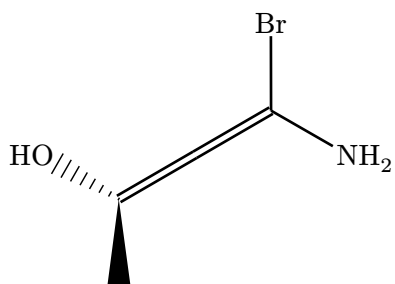
```

#let chiral-example(title, source) = align(center)[
  *#title*
  #v(2mm)
  #render-smiles(source, skeletal: true)
]

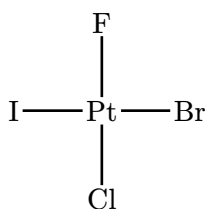
#grid(
  columns: 2,
  gutter: 8mm,
  row-gutter: 7mm,
  align: top,
  chiral-example([Allene · `@AL1`], "NC(Br)=[C@AL1]=C(O)C"),
  chiral-example([Square planar · `@SP2`], "[Pt@SP2](F)(Cl)(Br)I"),
  chiral-example([Trigonal bipyramidal · `@TB5`], "[As@TB5](F)(Cl)(Br)(N)S"),
  chiral-example([Octahedral · `@OH5`], "[Co@OH5](F)(Cl)(Br)(I)(N)S"),
)

```

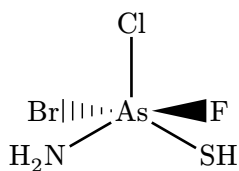
Allene · @AL1



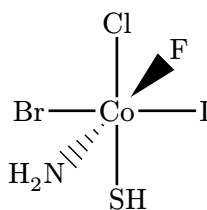
Square planar · @SP2



Trigonal bipyramidal · @TB5



Octahedral · @OH5



Part XIII

API Reference

XIII.1 Rendering Functions

```
#render-mol(  
  {data},  
  {record}: 1,  
  {abbreviate}: false,  
  {skeletal}: false,  
  {dump}: false,  
  {config}: {},  
  {annotations}: none,  
  {show-indices}: false,  
  {fidelity}: "ignore"  
) → content
```

Render a molecule from raw Molfile or SDF text.

The default layout: "avoid" adjusts spacing around usable input geometry. Use layout: "coordinates" for supplied spacing. Collapsed or numerically unstable coordinates receive a generated 2D layout while retaining source metadata and bond flags.

Argument

{data}

str | bytes | path

Raw .mol or .sdf data. Typst 0.15.0 and later may pass path(...) directly; older versions should pass read(...) output.

Argument

{record}: 1

int

One-based record number for multi-record SDF input.

Argument

{abbreviate}: false

bool

Enables abbreviated rendering.

Argument

{skeletal}: false

bool

Enables skeletal rendering. This overrides {abbreviate}.

Argument

{dump}: false

bool

Returns generated alchemist source code instead of rendering the molecule.

Argument
(**config**): (:) dictionary

Visual configuration for Alchemist and the renderer. See [Section VI](#) for layout, collision, and position options. The optional `ctfile` subdictionary is consumed by `molchemist` and accepts `highlight-paint`, `highlight-radius`, `highlight-thickness`, `highlight-label-padding`, `query-details`, `sgroup-stroke`, `sgroup-label-size`, `link-node-size`, and `variable-attachment-stroke`.

`highlight-radius` is the radius used for an unlabeled skeletal atom, `highlight-thickness` is the full width of a highlighted bond capsule, and `highlight-label-padding` expands the rendered glyph bounds before drawing a rounded rectangle. All three values scale with `(atom-sep)`.

Argument
(**annotations**): `none` annotation | array | none

Optional overlay annotation or array of annotations.

Argument
(**show-indices**): `false` bool | str

Debug overlay for annotation authoring. Use `true`, `"all"`, `"atoms"`, or `"bonds"`.

Argument
(**fidelity**): `"ignore"` str

Use `"strict"` to reject known remaining features that are preserved by inspection but not fully depicted. `"ignore"` keeps the normal rendering behavior.

#inspect-mol(**(data)**, (**record**): `1`) → dictionary

Parse one Molfile/SDF record into the versioned semantic IR without reducing it to drawing commands.

Argument
(**data**) str | bytes | path

Raw `.mol` or `.sdf` data. Typst 0.15.0 and later may pass `path(...)` directly; older versions should pass `read(...)` output.

Argument
(**record**): `1` int

One-based record number for multi-record SDF input.

The result includes `schemaVersion`, `format`, `header fields`, `atoms`, `bonds`, `stereoGroups`, `sgroups`, `linkNodes`, `collections`, `ordered properties`, `diagnostics`, and `rawRecord`. Atom and bond entries contain both a zero-based depiction index and a stable `sourceId`; atoms may contain `normalized query`, `rgroupLabels`, and `attachmentPoints`, bonds may contain `endpointSourceIds` and `attachmentMode`, and `SGroups` contain both `normalized kind` and

original typeCode plus structured SAP attachmentPoints. A collection contains its complete name, normalized kind, atom/bond/SGroup/3D/R-group member IDs, generic members, and exact rawEntry.

```
#render-smiles(  
  {smiles},  
  {abbreviate}: false,  
  {skeletal}: false,  
  {dump}: false,  
  {config}: (:),  
  {annotations}: none,  
  {show-indices}: false
```

```
) → content
```

Parse a SMILES string, generate 2D coordinates, and render the resulting molecule.

Argument	
{smiles}	str
A SMILES string.	

Argument	
{abbreviate}: false	bool
Enables abbreviated rendering after layout generation.	

Argument	
{skeletal}: false	bool
Enables skeletal rendering. This overrides {abbreviate}.	

Argument	
{dump}: false	bool
Returns generated alchemist source code instead of rendering the molecule.	

Argument	
{config}: (:	dictionary
Visual configuration for Alchemist and the renderer, including the options in Section VI .	

Argument	
{annotations}: none	annotation array none
Optional overlay annotation or array of annotations.	

Argument	
{show-indices}: false	bool str
Debug overlay for annotation authoring. Use true, "all", "atoms", or "bonds".	

XIII.2 MOL2 Functions

#render-mol2((data), (record): 1, ..(options)) → content

Render a Tripos MOL2 record. data accepts text, bytes, or a Typst 0.15.0+ path; record is one-based. Additional named options are forwarded to **#render-mol**, including skeletal, abbreviate, config, dump, annotations, show-indices, and fidelity.

#inspect-mol2((data), (record): 1) → dictionary

Inspect the selected MOL2 record after conversion to V3000. The return shape matches **#inspect-mol**; MOL2_SOURCE preserves the original selected record as a JSON string in its ordered SDF properties.

XIII.3 Reaction Functions

#inspect-reaction((data), (format): "reaction-smiles", (infer-mapping): false, (search-limit): 200000) → dictionary

Inspect reaction components, atom correspondence, bond changes, unmatched atoms, and search status. data accepts text, bytes, or a Typst 0.15.0+ path. format is "reaction-smiles" or "rxn". infer-mapping enables correspondence search; search-limit is a positive integer budget. Check ambiguous and searchComplete before treating inferred correspondence as unique.

#render-reaction(
 (data),
 (format): "reaction-smiles",
 (infer-mapping): false,
 (search-limit): 200000,
 (highlight-center): true,
 (mapping-policy): "report",
 (fidelity): "ignore",
 (conditions): none,
 (skeletal): true,
 (config): (:)
) → content

Render the reaction and retain its analysis in molchemist-reaction metadata. Input and search arguments match **#inspect-reaction**.

highlight-center controls bond-change highlighting. mapping-policy: "strict" rejects ambiguous or incomplete correspondence; "report" retains the analysis without rejecting it. fidelity: "strict" rejects unsupported CTfile depiction features in the components. conditions accepts content placed below the arrow, while agents appear above it. skeletal and config apply to the component drawings.

XIII.4 R-group Functions

#inspect-rgroup((data)) → dictionary

Inspect an RGfile supplied as text, bytes, or a Typst 0.15.0+ path. The result preserves root, members, raw logic, normalized conditions, and rawSource; members retain their group identifiers and attachment sites.

#render-rgroup(**{data}**, **{skeletal}**: **true**, **{fidelity}**: **"strict"**, **{config}**: **{:}**) → **content**

Render the root, all alternatives, attachment sites, and RLOGIC conditions. Input matches **#inspect-rgroup**. skeletal and config apply to each panel. fidelity accepts "strict" or "ignore"; strict rendering validates attachment properties in their R-group member context.

XIII.5 Anchor Helpers

#atom-anchor(**{index}**, **{anchor}**: **"mid"**) → **anchor**

Select an atom anchor for annotation placement.

Argument —
(index) **int**
Atom index shown by **{show-indices}**.

Argument —
(anchor): **"mid"** **str**
CeTZ anchor on the atom object, such as "north", "east", or "mid".

#bond-anchor(**{index}**, **{anchor}**: **"50%"**) → **anchor**

Select a bond anchor for annotation placement.

Argument —
(index) **int**
Bond index shown by **{show-indices}**.

Argument —
(anchor): **"50%"** **str**
CeTZ anchor on the bond object.

#molecule-anchor(**{anchor}**: **"center"**) → **anchor**

Select an anchor on the whole rendered molecule.

Argument —
(anchor): **"center"** **str**
CeTZ group anchor such as "center", "north", "east", "south", or "west".

#atom-ref(**{index}**) → **str**

Return the internal atom anchor name as a string.

#bond-ref(**{index}**) → **str**

Return the internal bond anchor name as a string.

XIII.6 Annotation Builders

```
#callout-annotation(
  {at},
  {label},
  {anchor}: "mid",
  {side}: "north-east",
  {label-at}: auto,
  {label-offset}: auto,
  {target-offset}: auto,
  {target-gap}: 0.2,
  {label-anchor}: auto,
  {leader}: "curve",
  {leader-start}: auto,
  {leader-end}: auto,
  {leader-points}: (),
  {leader-start-offset}: (0, 0),
  {leader-end-offset}: (0, 0),
  {mark}: none,
  {stroke}: 0.35pt + luma(40%),
  {label-size}: 0.82em,
  {label-gap}: 0.14,
  {label-inset}: 2.2pt,
  {label-radius}: 3pt,
  {label-fill}: luma(100%),
  {label-stroke}: 0.35pt + luma(70%),
  {boxed}: false,
  {name}: none
) → annotation
```

Build an external label with a leader line.

Argument	
{at}	anchor
Target anchor.	
Argument	
{label}	content
Label content.	
Argument	
{anchor}: "mid"	str
Anchor used to resolve {at}.	
Argument	
{side}: "north-east"	str

Preset label side. Supported values are "east", "west", "north", "south", and diagonal combinations.

Argument

{label-at}: **auto**

auto | dictionary

Explicit CeTZ coordinate for the label.

Argument

{leader}: **"curve"**

str

Leader style: "curve", "straight", or "elbow".

Argument

{leader-end}: **auto**

auto | dictionary

Manual CeTZ coordinate where the leader should stop.

Argument

{target-gap}: **0.2**

float

Clearance from the target when {leader-end} is automatic.

label-offset and target-offset adjust automatic placement. leader-start, leader-end, leader-points, leader-start-offset, and leader-end-offset provide exact routing. mark and stroke style the line. label-size, label-gap, label-inset, label-radius, label-fill, label-stroke, and boxed style the label; name assigns an optional CeTZ object name.

```
#arrow-annotation(
  {from},
  {to},
  {label}: none,
  {from-anchor}: "mid",
  {to-anchor}: "mid",
  {label-anchor}: "south",
  {label-offset}: (0, 0.45),
  {mark}: (end: ">>", fill: luma(0%)),
  {stroke}: luma(0%),
  {boxed}: false,
  {label-size}: 0.85em,
  {label-fill}: luma(100%),
  {label-stroke}: 0.35pt + luma(72%),
  {label-inset}: 2pt,
  {label-radius}: 2pt,
  {name}: none
) → annotation
```

Build a free arrow overlay. from-anchor and to-anchor resolve named endpoints; label-anchor and label-offset position the optional label. mark and stroke style the arrow. boxed,

label-size, label-fill, label-stroke, label-inset, and label-radius style its label; name assigns an optional CeTZ object name.

```
#Label-annotation(  
  {at},  
  {label},  
  {anchor}: "mid",  
  {label-anchor}: "south",  
  {offset}: (0, 0.45),  
  {boxed}: false,  
  {label-size}: 0.85em,  
  {label-fill}: luma(100%),  
  {label-stroke}: 0.35pt + luma(72%),  
  {label-inset}: 2pt,  
  {label-radius}: 2pt,  
  {name}: none  
)
```

→ **annotation**

Build a free text label overlay without a leader line. anchor, label-anchor, and offset control placement. boxed, label-size, label-fill, label-stroke, label-inset, and label-radius style the label; name assigns an optional CeTZ object name.

```
#cetz-annotation({body}, {name}: none) → annotation
```

Run custom CeTZ code after the molecule is drawn.

Argument

{body}

function

Function receiving the generated molecule name.

Argument

{name}: none

str | none

Optional name retained in the annotation descriptor.

Part XIV

Limitations

- Reaction correspondence can have multiple equally good solutions. Inspection reports ambiguity and search-budget exhaustion; strict mapping rejects those cases. Bond edits describe the supplied graph representations, not a reaction mechanism.
- User-defined and unknown internal COLLECTIONs have no standard default glyph and remain inspection-only. MDLV30/HILITE depicts atom and bond members and projects known SGroup membership onto their atoms and bonds. 3D-object, external R-group, generic and unknown-group members are preserved and reported by strict fidelity.
- Unknown future SGroup type codes are preserved for inspection but are not given an invented bracket convention; strict fidelity reports them. Multi-atom SUP groups without a usable label or with overlapping membership are preserved rather than contracted.
- M APO and ATTCHPT are R-group-member properties. Main-CTAB occurrences remain invalid in strict fidelity. Use `render - rgroup` to display them in their member context.
- Full mode can become crowded for large molecules because explicit hydrogens and atom labels occupy real page space. Prefer abbreviated or skeletal mode, or increase `{atom-sep}`.
- The default layout: "avoid" adjusts spacing without changing angles. Coincident vertices and labels centered on unrelated bonds can require layout: "reflow" or explicit atom positions.
- SMILES and unusable SDF coordinates are laid out with Coordgen. The result is deterministic for a given bundled plugin, but it may differ from an external chemical drawing program.
- Optional 3D projection infers nondegenerate tetrahedral orientation and near-planar double-bond geometry. It does not assign CIP descriptors or infer unspecified non-tetrahedral chirality.
- Extended-chirality layout uses branch rotation or a constrained whole-graph solution for cyclic ligands. Invalid topology or conflicting stereochemical constraints retain the source annotation.
- Coordinate mode automatically places atom/bond callouts and their leaders. Arbitrary CeTZ overlays, SGroup brackets and query-detail labels still require final figure review.

Part XV

License and Dependencies

The molchemist-authored Typst source is distributed under the MIT License. The published package also embeds precompiled WASM components, so its manifest uses the aggregate SPDX expression MIT AND BSD-3-Clause AND Apache-2.0 AND (Apache-2.0 WITH LLVM-exception). Molfile / SDF parsing is powered by `sdfrust`; SMILES parsing is based on `opensmiles`; SMILES 2D coordinate generation uses `CoordgenLibs`; rendering is handled by `alchemist` and `CeTZ`. See the distributed third-party notices for the complete file-to-license mapping and license texts.

The example SDF files and rendered example images are attributed separately from the package code. In particular, this manual includes PubChem-derived example structures such as [CID 241](#). See `THIRD_PARTY_NOTICES.md` and `docs/assets/README.md` for source URLs and the relevant NCBI data-usage policy.

Part XVI

Index

A

<code>anchor</code>	38
<code>annotation</code>	38
<code>#arrow-annotation</code>	44
<code>#atom-anchor</code>	42
<code>#atom-ref</code>	42

B

<code>#bond-anchor</code>	42
<code>#bond-ref</code>	42

C

<code>#callout-annotation</code>	43
<code>#cetz-annotation</code>	45

I

<code>#inspect-mol</code>	39
<code>#inspect-mol2</code>	41
<code>#inspect-reaction</code>	41
<code>#inspect-rgroup</code>	41

L

<code>#label-annotation</code>	45
--------------------------------------	----

M

<code>#molecule-anchor</code>	42
-------------------------------------	----

R

<code>#render-mol</code>	38
<code>#render-mol2</code>	41
<code>#render-reaction</code>	41
<code>#render-rgroup</code>	42
<code>#render-smiles</code>	40